Relatório técnico — Restaurant API (Fase 2)

1. Objetivo

Atender ao Tech Challenge da fase 2: CRUD de tipo de usuário (com associação a usuários), cadastro de restaurantes e itens do cardápio, organizados em Clean Architecture, com documentação, collection de testes, Docker Compose e cobertura de testes.

2. Arquitetura

Camadas (`com.api.restaurant`):

Camada	Responsabilidade
`domain`	Modelos, enums, ports e exceções de negócio
`application`	Use cases / services
`infrastructure`	JPA (entities, adapters), Security, OpenAPI, seed
`presentation`	Controllers REST, DTOs HTTP, ProblemDetail

Detalhes e checklist: CLEAN-ARCHITECTURE.md.

Fluxo típico: `Controller` → `*Service` → port (`*Repository`) ← adapter JPA.

3. Modelo de dados (resumo)

- **UserType**: `name` (único) + `category` (`CUSTOMER` | `RESTAURANT_STAFF`)
- **User**: dados de conta + `userTypeId`
- **Restaurant**: nome, endereço, tipo de cozinha, horários (`dayOfWeek` / `opensAt` / `closesAt`), `ownerId`
- **MenuItem**: nome, descrição, preço, `dineInOnly` , `photoPath` , `restaurantId`

Regras principais

- Excluir tipo bloqueado se houver usuários vinculados.
- Trocar tipo do usuário bloqueado se ele for dono de restaurante(s).
- Criar/atualizar restaurante exige dono com categoria `RESTAURANT_STAFF` .
- Excluir restaurante remove itens do cardápio (cascade).
- `dineInOnly=true` → só consumo no local; `false` → pode pedir para fora.

4. API

Base: `/api/v1`. Erros: **ProblemDetail (RFC 7807)**.

Exemplos:

POST `/api/v1/user-types`

```
{ "name": "Gerente", "category": "RESTAURANT_STAFF" }
```

POST `/api/v1/restaurants`

```
{
  "name": "Cantina",
  "address": {
    "zipCode": "01310-100",
    "street": "Av Paulista",
    "number": "1000",
    "city": "São Paulo",
    "state": "SP"
  },
  "cuisineType": "Italiana",
  "openingHours": [
    { "dayOfWeek": "MONDAY", "opensAt": "11:00:00", "closesAt": "22:00:00" }
  ],
  "ownerId": 1
}
```

POST `/api/v1/restaurants/{id}/menu-items`

```
{
  "name": "Lasanha",
  "description": "Bolonhesa",
  "price": 39.90,
  "dineInOnly": true,
  "photoPath": "/photos/lasanha.jpg"
}
```

Documentação interativa: Swagger UI em `/swagger-ui/index.html`.

5. Infraestrutura

- `compose.yaml`: Postgres (`5436→5432`) + profile `full` para a aplicação.
- `Dockerfile`: multi-stage (JDK 25 build / JRE run).
- Variáveis do serviço `app`: `SPRING_DATASOURCE_*`, `SPRING_DOCKER_COMPOSE_ENABLED=false`.

6. Qualidade

- Testes unitários dos services (Mockito).
- Teste de integração HTTP (`MockMvc` + H2).
- JaCoCo no `verify` com mínimo de 80% de linhas (excluídos DTOs, config, entities JPA e classe main).

7. Como reproduzir

Ver README.md. Collection Postman na raiz do repositório.

8. Vídeo de entrega

Demonstração da solução: https://www.loom.com/share/e3c2d23858724deba5c3a47c48c92fa5